

Module: Big Data Analytics Technologies (Postgraduate Coursework)

Assignment Duration: 4 Weeks

Module Learning Outcomes Assessed: Knowledge of modern big data architectures, real-time streaming topologies, distributed analytics engines, graph structures, and strategic data governance frameworks.

Deployment Constraint: All practical tasks MUST use Docker / Docker Compose containers. Native host OS dependencies are prohibited.

Each question has a weight of 20 marks out of 100.

Task 1: Distributed Time-Series Data Management using InfluxDB (week 1)

Objective: Design, provision, and evaluate a high-throughput time-series database architecture using InfluxDB inside a containerized topology. Students will master time-series schema structuring, bucket retention optimization, and functional metric querying.

Dataset Link: https://snap.uaf.edu/tools/community-charts/data/fairbanks_climate.csv

Step-by-Step Implementation Activity

Step 1.1: Containerized Environment Provisioning

Construct a docker-compose.yml file configuring an autonomous InfluxDB v2.x container. Expose the API and user interface port 8086. Configure persistent storage bindings mapping your host local directory to the internal data path /var/lib/influxdb2. Set environment parameters to automatically bootstrap organization and bucket structures upon execution.

Step 1.2: Database Schema & Ingestion Scripting

Develop an automated data ingest pipeline using Python (official influxdb-client package) or an engineered Shell/curl routine. Parse the public climate CSV dataset line-by-line, translate rows into InfluxDB Line Protocol formatting, and inject them into the database while properly mapping the records' original historical timestamps rather than the local deployment system clock.

Step 1.3: Flux Time-Series Analytical Querying

Write and execute three specialized analytical operations using the functional data scripting language Flux via the system CLI or web panel: (1) Window Aggregation to compute sliding hourly averages across an extended observation span; (2) Anomaly Isolation filters to capture observations exceeding two standard deviations from the dataset mean; and (3) Downsampling tasks that continuously summarize historical data into an auxiliary bucket governed by an explicit 30-day retention rule.

Task 2: Real-Time Stream Ingestion & Processing with Apache Kafka and Apache Flink (week 2)

Objective: Construct an end-to-end distributed stream processing infrastructure. Students will implement live telemetry data pipelines, feed streams into multi-partition Apache Kafka topics, and execute stateful, windowed compute matrices inside Apache Flink.

Dataset Link: https://data.austintexas.gov/Transportation-and-Mobility/Camera-Traffic-Counts/sh59-i6y9/about_data

Step-by-Step Implementation Activity

Step 2.1: Multi-Container Orchestration Setup

Design a cohesive docker-compose.yml landscape initializing a fully containerized real-time network layer. Include at least one distributed Apache Kafka cluster node (leveraging modern KRaft mode or a companion ZooKeeper daemon), along with an active Apache Flink JobManager node and an allocated Flink TaskManager node linked over a shared virtual network bridge.

Step 2.2: Kafka Topic Architecture & Live Simulation

Access the Kafka container execution context and declare an optimized message channel named 'traffic-telemetry' built with 3 distinct partitions and a replication factor of 1. Code an automated producer application to iteratively ingest rows from the open-source Austin camera metadata, parse them into structured JSON elements, and feed them into the Kafka broker every 2 seconds to simulate true stream concurrency.

Step 2.3: Stateful Flink Processing & Out-of-Order Handling

Write a complete Flink stream parsing bundle using Java or PyFlink. Configure an explicit Bounded-OutOfOrderness Watermarking strategy allowing up to a 10-second skew window for out-of-sequence event reconciliation. Orchestrate a 10-minute tumbling window operation to output a moving total of vehicle counts by tracking sensor points, and deploy your pipeline through the Flink console dashboard.

Task 3: Scalable Data Analytics with Apache Spark (week 3)

Objective: Utilize the parallel computing model of Apache Spark to build robust Extract-Transform-Load (ETL) paths, handle large-scale dataset ingest, and execute deep network topology transformations over clustered compute nodes.

Dataset Link: <https://snap.stanford.edu/data/web-BerkStan.html>

Step-by-Step Implementation Activity

Step 3.1: Spark Container Environment Setup

Build out a local target standalone Spark computation ring within your Docker Compose blueprint. Provision one Master engine node (exposing administration UI dashboard on port 8080) and link two independent Spark Worker instances. Constrain each worker's available performance profiles explicitly (e.g., maximum 2 compute cores and 2GB RAM allocation) and connect a shared host storage layer containing the uncompressed SNAP network text files.

Step 3.2: PySpark Distributed Job Implementation

Construct a Python PySpark engineering script to manage the file collection: (1) Use lazy evaluation principles to cleanly parse the text file into a functional Spark Dataframe while striping out metadata headers; (2) Group and aggregate target vertex indexes to calculate structural in-degree distributions; and (3) Order rows to identify the Top 50 dominant destination nodes while adding explicit memory optimizations such as dataframe caching or broadcast joins.

Step 3.3: Execution Metrics Capture

Submit the script package utilizing the master spark-submit executable context inside the active cluster. Review and extract performance records from the Spark Web Console, logging execution Stage run durations, the full Directed Acyclic Graph (DAG) logic chain, Shuffle space allocation metrics, and any signs of data skew across worker groups.

Task 4: Graph Database Engineering using Neo4j (week 4 1st half)

Objective: Develop graph-native storage structures using Neo4j. Students will conceptualize dense relational graph networks, compose optimized Cypher querying scripts, and run performance-driven node lookups without incurring standard SQL multi-join computational penalties.

Dataset Link: <https://snap.stanford.edu/data/cit-Patents.html>

Step-by-Step Implementation Activity

Step 4.1: Neo4j Docker Provisioning

Instantiate a containerized environment running a Neo4j Community Edition engine using basic Docker commands or an orchestration configuration. Configure exposed boundaries across port 7474 for standard web traffic and port 7687 for internal Bolt communication protocol. Secure the node using custom credential environment flags and verify browser access via localhost.

Step 4.2: Cypher Graph Ingestion

Extract a focused subset of the first 5,000 document citation paths from the downloaded open SNAP patent text structure. Create a bulk import logic loop inside Cypher utilizing standard LOAD CSV procedures or automated file injection utilities to convert raw textual records into structured graph node sets connected by explicit directed relationships.

Step 4.3: Graph Querying and Network Analytics

Formulate and run three descriptive, structural graph analysis queries using the Cypher query dialect, documenting execution behavior: (1) Identify direct neighbor layers matching specific target reference items; (2) Compute degree centrality to discover top-tier patent records with large volumes of incoming citation branches; and (3) Execute shortest path search routines between distant node points to map hidden reference linkages.

Task 5: Theoretical Foundations & Future Engineering Trends (week 4^{2nd} half)

Objective: Conduct an intensive academic literature review and analytical investigation into enterprise data governance and advanced computing paradigms. This component must be written in formal academic style and properly cite a minimum of five recent peer-reviewed research papers.

Part 5.1: Enterprise Data Governance Frameworks

Provide a critical, theoretical synthesis evaluating enterprise Data Governance methodologies. Contrast the functional limitations of highly centralized architectural patterns (e.g., traditional Data Lakes, corporate Data Warehouses) against modern distributed frameworks like Data Mesh and Data Contracts. Detail how modern regulations (such as GDPR, CCPA) govern the lifecycle of big data engineering pipelines. Explain the operational roles played by metadata catalog systems (e.g., Apache Atlas, OpenMetadata) in tracing automated data lineage and access controls.

Part 5.2: Emerging Frontiers – Quantum Computing & LLM-Driven Loop Engineering

Analyze the operational disruption of advanced technical computational layers:

1. Quantum Big Data Paradigms: Explain how Quantum Machine Learning (QML) and Quantum-Enhanced Databases challenge standard constraints on NP-hard processing problems, cryptographic signatures, and multidimensional indexing optimizations.
2. LLM Loop Engineering: Investigate the emerging practice of utilizing Large Language Models within closed-loop data engineering cycles (e.g., automated code-generation agents, predictive pipeline optimization, autonomous self-healing ETL errors, and structural text-to-SQL layer synthesis).
3. Open Synthesis: Evaluate the system-level bottlenecks that occur when integrating nondeterministic LLM outputs with strict, deterministic distributed runtime engines (such as Spark or Flink).